

Segunda Lista de Engenharia de Software RT

Capítulo 2 — Processos de Software

Ian Sommerville — *Software Engineering*, 10^a ed.

Trabalho Individual

Questão 1

Enunciado

Justificando sua resposta com base no tipo de sistema a ser desenvolvido, sugira o modelo genérico de processo de software mais adequado para ser usado como base para a gerência do desenvolvimento dos sistemas a seguir:

- Um sistema para controlar o antibloqueio de frenagem de um carro (ABS).
- Um sistema de realidade virtual para dar apoio à manutenção de software.
- Um sistema de contabilidade para uma universidade, que substitua um sistema já existente.
- Um sistema interativo de planejamento de viagens que ajude os usuários a planejar viagens com menor impacto ambiental.

Resposta

(a) Sistema ABS — Modelo em Cascata (Waterfall)

Um sistema de antibloqueio de frenagem é um **sistema crítico de tempo real** embarcado com requisitos de segurança altamente rigorosos e definidos antes do início do desenvolvimento. Falhas podem custar vidas. Por isso, exige:

- Especificação completa e formal de requisitos antes da implementação;
- Validação e verificação formais em cada fase;
- Rastreabilidade total entre requisitos, projeto, código e testes.

O **modelo em cascata** é o mais adequado porque impõe uma sequência rígida de fases (requisitos → projeto → implementação → verificação → manutenção) com saídas documentadas em cada etapa, atendendo a padrões como IEC 61508 (segurança funcional). A completeza dos requisitos antes da codificação é viável pois os requisitos físicos do sistema são bem compreendidos e estáveis.

(b) Sistema de Realidade Virtual para Manutenção de Software — Desenvolvimento Evolucionário / Espiral

Sistemas de realidade virtual são inerentemente exploratórios: as interfaces, metáforas de interação e as necessidades dos usuários só ficam claras na prática. Os requisitos *não podem ser completamente especificados* antecipada-

mente.

O **modelo evolucionário** (ou o **modelo espiral de Boehm**) é mais adequado porque:

- Permite protótipos sucessivos com avaliação de usuários a cada ciclo;
- Incorpora gerência de riscos explícita (fundamental em tecnologia nova);
- Evoluções do sistema são esperadas conforme os usuários descobrem novas necessidades.

(c) Sistema de Contabilidade Universitário (substituição) — Desenvolvimento Baseado em Reuso / Incremental

Como já existe um sistema legado em operação, os requisitos são relativamente conhecidos (mesmo fluxo contábil) e há grande disponibilidade de componentes reutilizáveis (frameworks ERP, módulos de relatórios fiscais, bibliotecas contábeis). O risco principal é a migração de dados e a aceitação dos usuários.

O **modelo baseado em reuso** é o mais adequado porque:

- Maximiza o uso de software pronto (reduz custo e prazo);
- Entregas incrementais permitem migração gradual com menor risco operacional;
- Requisitos derivam do sistema existente, minimizando incertezas.

(d) Sistema Interativo de Planejamento de Viagens — Desenvolvimento Incremental / Ágil

Este é um sistema voltado a usuários finais em um domínio em constante evolução (regulamentações ambientais, bases de dados de emissões, preferências de viajantes). Os requisitos mudarão ao longo do desenvolvimento conforme os usuários interagem com o produto.

O **desenvolvimento incremental** (base para métodos ágeis) é o mais adequado:

- Entregas frequentes de funcionalidades priorizadas;
- Feedback contínuo dos usuários incorporado em cada incremento;
- Facilidade de adaptação a novas fontes de dados ambientais ou regulamentações.

Questão 2

Enunciado

Explique por que o desenvolvimento incremental é o método mais eficaz para o desenvolvimento de sistemas de software de negócios. Por que esse modelo é menos adequado para a engenharia de sistemas de tempo real?

Resposta

Eficácia no desenvolvimento de sistemas de negócios:

Sistemas de negócios operam em ambientes dinâmicos onde requisitos mudam

continuamente em resposta a alterações regulatórias, de mercado e organizacionais. O desenvolvimento incremental é mais eficaz porque:

1. **Entrega de valor antecipada:** versões funcionais do sistema são disponibilizadas cedo, gerando retorno sobre o investimento antes da conclusão total do projeto.
2. **Validação contínua:** usuários de negócio verificam cada incremento e fornecem feedback, reduzindo o risco de o sistema final não atender às necessidades reais.
3. **Acomodação de mudanças:** como os requisitos de negócio evoluem, incrementos futuros absorvem as mudanças sem comprometer o que já foi entregue.
4. **Redução do risco financeiro:** problemas são detectados cedo, quando o custo de correção ainda é baixo.
5. **Gerenciamento do escopo:** funcionalidades prioritárias são entregues primeiro, garantindo que o sistema seja útil mesmo se o projeto não for concluído integralmente.

Inadequação para sistemas de tempo real:

Sistemas de tempo real (controle industrial, aviônicos, sistemas médicos) têm **requisitos não-funcionais críticos** — latência, taxa de resposta, determinismo temporal — que não podem ser validados em incrementos isolados:

1. **Integração indivisível:** o comportamento temporal emergente só pode ser verificado quando todos os componentes estão integrados. Um incremento parcial pode satisfazer requisitos funcionais mas violar garantias de tempo.
2. **Certificação formal:** normas de segurança (DO-178C, IEC 61508) exigem documentação e rastreabilidade completas desde os requisitos, o que conflita com a natureza evolutiva do modelo incremental.
3. **Hardware acoplado:** o comportamento de tempo real depende da interação precisa entre software e hardware; mudanças incrementais no software podem invalidar análises de *worst-case execution time*.
4. **Custo de falha catastrófico:** em sistemas críticos, uma falha entregue em produção pode custar vidas, inviabilizando a entrega precoce de versões incompletas.

Questão 3

Enunciado

Considere o modelo de processo baseado em reuso da Figura 2.3. Explique por que, nesse processo, é essencial ter duas atividades distintas de engenharia de requisitos.

Resposta

No modelo baseado em reuso, o processo possui duas atividades de engenharia de requisitos:

1. **Especificação de Requisitos Iniciais** — captura o que o cliente *realmente precisa*, sem considerar o que está disponível para reuso. É necessária para estabelecer uma linha de base das necessidades do sistema.
2. **Modificação de Requisitos** — após a descoberta e avaliação dos componentes reutilizáveis disponíveis, os requisitos originais são *revisados e adaptados* para refletir o que os componentes conseguem oferecer.

Por que as duas atividades são essenciais:

A primeira especificação (independente de reuso) é necessária porque os componentes disponíveis são descobertos *após* a elicitação inicial. Sem ela, não é possível avaliar quais requisitos podem ser satisfeitos por reuso e quais precisam de desenvolvimento novo.

A segunda atividade existe porque os componentes reutilizados raramente satisfazem os requisitos originais **exatamente**. Há um *gap* entre o que o cliente deseja e o que o componente oferece. Ajustar os requisitos ao invés de forçar a adaptação do componente preserva a integridade arquitetural e reduz custos de manutenção. Sem essa segunda fase, o sistema seria desenvolvido com discrepâncias não documentadas entre requisitos e funcionalidade real, gerando insatisfação do cliente e dívida técnica.

Nota

Sommerville destaca que a principal desvantagem do modelo baseado em reuso é exatamente essa negociação de requisitos: o cliente pode precisar aceitar um sistema que não atende completamente suas necessidades originais.

Questão 4

Enunciado

Sugira porque é importante, no processo de engenharia de requisitos, fazer uma distinção entre desenvolvimento dos requisitos do usuário e desenvolvimento de requisitos de sistema.

Resposta

Requisitos do usuário descrevem, em linguagem natural acessível, o que os usuários e demais *stakeholders* esperam que o sistema faça para apoiar seu trabalho. São **orientados a objetivos** e independentes de como o sistema será implementado.

Requisitos de sistema são especificações técnicas detalhadas, precisas e completas de *como* o sistema deve realizar o que foi pedido, incluindo restrições de desempenho, protocolos, interfaces, segurança e consistência de dados.

Por que a distinção é importante:

1. **Audiências diferentes:** gerentes, clientes e usuários finais leem os requisitos de usuário para validar que o sistema resolve seus problemas; arquitetos e desenvolvedores leem os requisitos de sistema para implementá-lo. Misturar os dois níveis confunde cada grupo.
2. **Comprometimento contratual:** os requisitos de usuário costumam compor o contrato entre cliente e fornecedor. Requisitos de sistema são internos à equipe de desenvolvimento. Confundir os dois pode gerar disputas sobre o que foi contratado.
3. **Rastreabilidade:** a separação permite rastrear cada requisito de sistema a um ou mais requisitos de usuário, facilitando a verificação de que todos os objetivos do cliente foram atendidos.
4. **Gerenciamento de mudanças:** mudanças em requisitos de usuário têm impacto diferente de mudanças em requisitos de sistema. Separar os dois permite avaliar o impacto de cada mudança com precisão.
5. **Detalhamento progressivo:** permite um processo de engenharia gradual — primeiro valida-se o *o quê* com os stakeholders, depois especifica-se o *como* com a equipe técnica, evitando refazer trabalho técnico por mal entendimento de objetivos.

Questão 5

Enunciado

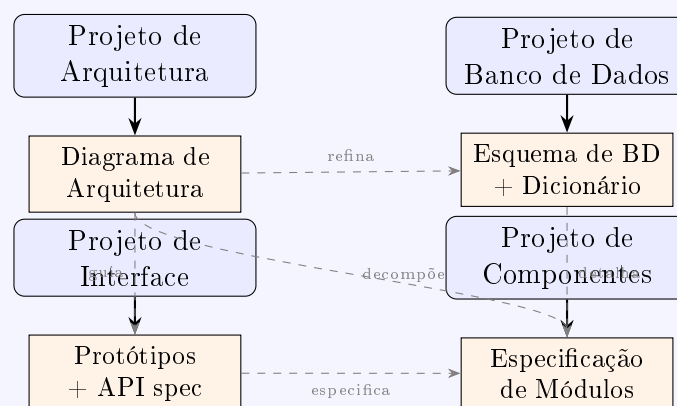
Descreva as principais atividades do processo de projeto de software e as saídas dessas atividades. Usando um diagrama, mostre as possíveis relações entre as saídas dessas atividades.

Resposta

As principais atividades do processo de projeto de software, conforme Sommerville, são:

Atividade	Saída
Projeto de arquitetura	Diagrama de componentes/módulos; estilo arquitetural (camadas, cliente-servidor, MVC etc.)
Projeto de banco de dados	Modelo entidade-relacionamento; esquema lógico/físico; dicionário de dados
Projeto de interface	Protótipos de telas; descrição de fluxo de navegação; especificação de API/protocolo de comunicação
Projeto de componentes	Especificação detalhada de cada módulo; algoritmos; pré/pós-condições; pseudocódigo ou UML de classe

Diagrama de relações entre as saídas:



O **diagrama de arquitetura** é a saída central: define os subsistemas e suas interfaces, orientando todas as demais atividades. O projeto de banco de dados refina a arquitetura de dados; o projeto de interface especifica os contratos entre componentes; e o projeto de componentes detalha cada módulo individualmente. Todas as saídas se integram na documentação final de projeto que orienta a fase de implementação.

Questão 6

Enunciado

Explique por que, em sistemas complexos, as mudanças são inevitáveis. Exemplifique as atividades de processo de software que ajudam a prever as mudanças e fazer com que o software seja desenvolvido mais tolerante a mudanças (desconsidere prototipação e entrega incremental).

Resposta

Por que mudanças são inevitáveis em sistemas complexos:

1. **Ambiente externo mutável:** legislação, regulamentações, condições de mercado e tecnologias de suporte mudam independentemente da vontade do time de desenvolvimento.
2. **Requisitos mal compreendidos:** em sistemas complexos, os stakeholders frequentemente só descobrem o que realmente precisam após ver o sistema em operação.
3. **Conflito de interesses:** diferentes stakeholders têm visões conflitantes que só emergem durante o desenvolvimento.
4. **Evolução do negócio:** a organização que solicitou o sistema pode se reestruturar, mudar de estratégia ou ser adquirida durante o projeto.
5. **Descoberta de falhas:** erros nos requisitos e no projeto só são revelados em fases posteriores, exigindo revisões retroativas.

Atividades que ajudam a prever e tolerar mudanças (excluindo prototipação e entrega incremental):

- **Antecipação de mudanças (prevenção):**

- *Revisões de requisitos (requirements reviews):* análise sistemática dos requisitos com os stakeholders para identificar inconsistências, ambiguidades e itens provavelmente instáveis, antes da implementação.
- *Rastreabilidade de requisitos:* manter matrizes de rastreabilidade entre requisitos e artefatos de projeto permite avaliar o impacto de mudanças antes de implementá-las.

- **Tolerância a mudanças (acomodação):**

- *Projeto orientado a mudanças (change-oriented design):* uso de princípios como alta coesão, baixo acoplamento e ocultamento de informação (*information hiding*) isola módulos, de modo que uma mudança em um componente não propague efeitos para outros.
- *Gerenciamento de configuração:* controle de versões, baselines e gestão de mudanças formais garantem que alterações sejam rastreadas e revertidas se necessário, sem comprometer a integridade do sistema.

Questão 7

Enunciado

Explique por que os sistemas desenvolvidos como protótipos normalmente não devem ser usados como sistemas de produção.

Resposta

Protótipos são construídos sob restrições fundamentalmente diferentes das de um sistema de produção:

1. **Requisitos não-funcionais ignorados:** protótipos priorizam demonstrar funcionalidades visíveis. Desempenho, segurança, disponibilidade, escalabilidade e tolerância a falhas são deliberadamente omitidos para acelerar o desenvolvimento.
2. **Dívida técnica intencional:** atalhos de implementação, ausência de tratamento de erros, hardcoding de valores e falta de modularização são aceitáveis em protótipos mas comprometem a manutenibilidade e confiabilidade em produção.
3. **Ausência de testes rigorosos:** um protótipo raramente passa por ciclos completos de testes de sistema, regressão e segurança. Usá-lo em produção expõe os usuários a falhas não descobertas.
4. **Tecnologias inadequadas:** linguagens, frameworks ou bancos de dados escolhidos pela velocidade de prototipagem podem não ser adequados para escala, manutenção de longo prazo ou integração com sistemas legados.
5. **Ausência de documentação:** protótipos carecem de documentação de arquitetura, requisitos rastreáveis e documentação de manutenção, tornando a evolução futura extremamente custosa.
6. **Pressão organizacional indevida:** transformar um protótipo em sistema de produção cria uma *proto-trap* — a organização fica presa a decisões técnicas ruins tomadas sob pressão de prazo, pagando juros de dívida técnica indefinidamente.

A conclusão de Sommerville é que protótipos devem ser **descartados** após cumprir seu propósito de elicitação/validação de requisitos, e o sistema real deve ser desenvolvido do zero com base nos aprendizados obtidos.

Questão 8

Enunciado

Explique por que o modelo em espiral de Boehm é um modelo adaptável que apoia tanto as atividades de prevenção de mudanças quanto as de tolerância a mudanças. Na prática, esse modelo não tem sido amplamente usado. Sugira as possíveis razões para isso.

Resposta

O modelo espiral como modelo adaptável:

O modelo espiral de Boehm organiza o processo em **ciclos iterativos**, cada um com quatro quadrantes: (1) definição de objetivos e alternativas, (2) *avaliação de riscos e prototipação*, (3) desenvolvimento e validação, (4) planejamento da próxima iteração.

- **Prevenção de mudanças:** a análise explícita de riscos em cada ciclo identifica antecipadamente áreas de incerteza (técnica, de requisitos, organizacional). Prototipação e avaliação antecipam problemas antes de grandes investimentos, funcionando como atividade de “antecipação de mudanças”.
- **Tolerância a mudanças:** a estrutura iterativa permite que cada ciclo incorpore mudanças descobertas no ciclo anterior. Não há comprometimento irreversível com uma arquitetura específica no início do projeto; o design evolui com o crescente entendimento do problema.

Por que o modelo espiral não é amplamente adotado na prática:

1. **Complexidade de gestão:** cada ciclo exige análise de riscos formal por profissionais especializados. Poucos times têm expertise em gerenciamento de riscos quantitativo, e a documentação necessária é extensa.
2. **Custo e overhead:** a cerimônia de cada ciclo (documentação, revisão de riscos, planejamento) é custosa para projetos pequenos ou médios, nos quais a relação custo-benefício não se justifica.
3. **Dificuldade contratual:** contratos de desenvolvimento de software geralmente exigem escopo, prazo e custo fixos. O modelo espiral pressupõe escopo emergente, o que é incompatível com contratos de preço fixo.
4. **Surgimento de métodos ágeis:** a partir dos anos 2000, métodos ágeis (Scrum, XP, Kanban) ofereceram adaptabilidade similar com muito menos overhead, tornando o modelo espiral menos atrativo.
5. **Falta de ferramentas de suporte:** não há ferramentas amplamente adotadas que automatizem ou guiem a avaliação de riscos periódica exigida pelo modelo espiral.

Questão 9

Enunciado

Quais são as vantagens de proporcionar visões estáticas e dinâmicas do processo de software, assim como no Rational Unified Process?

Resposta

O **Rational Unified Process (RUP)** oferece simultaneamente:

- **Visão dinâmica (eixo temporal):** organizada em quatro fases — Iniciação, Elaboração, Construção e Transição — que descrevem *quando* as atividades ocorrem ao longo do projeto, com marcos e objetivos específicos em cada fase.
- **Visão estática (fluxos de trabalho):** organizada em disciplinas — Requisitos, Análise e Projeto, Implementação, Testes, Implantação etc. — que descrevem *o que* é feito, independentemente do tempo.

Vantagens da combinação de ambas as visões:

1. **Planejamento e execução alinhados:** a visão dinâmica permite ao gerente de projeto planejar cronogramas e marcos. A visão estática garante que os engenheiros saibam quais práticas aplicar em cada disciplina.
2. **Comunicação clara com diferentes audiências:** gestores entendem o processo pelo eixo temporal (fases, entregas); engenheiros o entendem pelo eixo das disciplinas (o que fazer e como). As duas visões atendem a ambos sem perda de informação.
3. **Adaptabilidade controlada:** todas as disciplinas ocorrem em todas as fases, mas com intensidade variável. Isso deixa explícito que testes, por exemplo, não são atividades apenas do final do projeto, sem perder a estrutura temporal que orienta prioridades.
4. **Rastreabilidade:** a intersecção fase $\times$ disciplina permite identificar com precisão quando e onde artefatos específicos devem ser produzidos, facilitando auditorias e revisões.
5. **Suporte a iteratividade:** as fases dinâmicas acomodam iterações dentro de cada fase, enquanto as disciplinas estáticas garantem que boas práticas de engenharia sejam aplicadas consistentemente em cada iteração, independentemente do ponto do projeto.

Questão 10

Enunciado

Historicamente, a introdução de tecnologia provocou mudanças profundas no mercado de trabalho e, pelo menos temporariamente, deixou muitas pessoas desempregadas. Discuta se a introdução da automação extensiva em processos pode vir a ter as mesmas consequências para os engenheiros de software. Se sua resposta for não, justifique. Se você acha que sim, que vai reduzir as oportunidades de emprego, é ética a resistência passiva ou ativa, pelos engenheiros afetados, a introdução dessa tecnologia?

Resposta

A automação do processo de software e o impacto sobre o emprego:

A resposta mais honesta é **sim, parcialmente**. A automação extensiva — incluindo ferramentas de geração de código por IA, testes automatizados, integração contínua e plataformas *low-code/no-code* — tende a deslocar uma parte significativa de tarefas de engenharia de software, especialmente as mais rotineiras:

- Codificação de lógica de negócio simples e bem especificada;
- Geração de testes unitários e de integração;
- Refatoração e modernização de código legado;
- Desenvolvimento de interfaces padrão (CRUD, dashboards).

No entanto, a automação **não elimina** a necessidade de engenheiros para:

- Elicitar e interpretar requisitos complexos e ambíguos dos stakeholders;
- Tomar decisões arquiteturais com implicações de longo prazo;
- Garantir segurança, privacidade e conformidade regulatória;
- Validar criticamente a saída das ferramentas de IA (que cometem erros sistemáticos não óbvios);
- Sistemas críticos onde verificação formal é obrigatória.

O efeito mais provável é uma **transformação da profissão**, similar ao que ocorreu com outras disciplinas de engenharia com a chegada de ferramentas CAD: redução de papéis operacionais e elevação da demanda por papéis de julgamento, supervisão e domínio especializado.

Sobre a ética da resistência:

A resistência — passiva ou ativa — pelos engenheiros afetados é compreensível do ponto de vista humano, mas é **eticamente problemática** quando prejudica terceiros ou o bem público:

- **Resistência passiva (sabotagem sutil, atraso intencional):** viola os princípios de ética profissional de engenharia, que incluem a responsabilidade de agir no melhor interesse do público e dos empregadores. Atrasar a adoção de tecnologias que aumentam qualidade ou reduzem custos prejudica usuários e organizações.
- **Resistência ativa (lobby, organização sindical, pressão legal):** é **eticamente legítima** quando exercida dentro dos marcos legais e democráticos. Assim como trabalhadores industriais têm o direito de negociar

condições de transição, engenheiros têm o direito de exigir políticas de requalificação, aviso prévio e reconhecimento da perda de renda, desde que não prejudiquem a segurança dos sistemas ou os usuários finais.

Em suma: adaptar-se, redirecionar esforços para atividades de maior valor agregado e participar ativamente da discussão sobre políticas de transição é mais ético e eficaz do que resistir à mudança tecnológica per se.

Referência: Ian Sommerville. **Software Engineering**, 10^a ed. Pearson, 2016. Capítulo 2: Software Processes.